

Precedence

Mathematical functions assume that multiplication takes **precedence** over addition. *SimTalk* can combine mathematical as well as relational operators. The given precedence of the operators spares you the effort of having to insert too many parentheses. You only have to set them if you would like to deviate from that given order.

Remarks

Plant Simulation executes an expression within parentheses first. Within the parentheses, the precedence rules do apply. For parentheses within parentheses, *Plant Simulation* analyzes the innermost expression first.

The table lists the **precedence of operators**. Operators with the same precedence are listed in the same line. The highest priority is listed first.

Precedence	Operator	Description
highest	()	parenthesis
	– NOT	leading sign, negation
	*, /, //, \	multiplication, division, integer division, modulo
	+, –	addition, subtraction
	<, <=, =, !=, >=, >	less-than, less than or equal, equal, not equal, greater-than equal, greater-than
	AND	logic AND
	OR	logic Or
lowest	:=	assignment

Control Flow Statements

Control Flow Statements

A control is not always executed linearly. Different events influence the course of execution a control takes and require an appropriate reaction. *Plant Simulation* provides various language constructs for programming control flow statements.

- **Branching with if-else-end**
- **Multiple Branching with if-elseif-end**
- **Conditional Expressions with when-then-else**
- **Check for a Large Amount of Different Values with switch [SimTalk]**

- [Loops \[SimTalk\]](#)
- [waituntil \[SimTalk\]](#) and [stopuntil \[SimTalk\]](#) for suspending *Methods*

Branching with if-else-end

The `if`-statement uses the given condition to decide where to continue: Either with the following statements or with the statements in the `else` branch. Branching is terminated with the keyword `end`.

Remarks

Plant Simulation executes the `statement_list1` if the condition is *true*, otherwise it executes `statement_list2` after `else`. If you did not program the `else` branch because you do not need it, *Plant Simulation* continues the execution with the first statement after branching.

A `statement_list` consists of assignments, method calls, loops, or additional branches. There is no nesting limit.

Syntax

```
if condition
  statement_list1
[else
  statement_list2]
end
```

Examples with an else branch

Within the condition *Plant Simulation* checks if the *Transporter* has reached its destination (`@.destination = current`). If the condition is *true*, the control transfers a *MU* to the workstation. If the condition is *false*, the *Transporter* moves on.

```
// Is this delivery destined for us?
if @.destination = current // yes, then unload
  @.Cont.move(workstation)
else // if not, then move on
  @.move(current.succ(1))
end
```

```
var ages := [34,42,18,44,53,12,63]
var countBelow30: integer
var countAbove29: integer
for var i := 1 to ages.dim
  if ages[i] < 30
    countBelow30 := countBelow30 + 1
  else
    countAbove29 := countAbove29 + 1
  end
next
```

```
print "Below 30: ", countBelow30  
print "Above 29: ", countAbove29
```

Example without an else branch

The condition of the branching returns *true* when *MUs* are located on the *Transporter*. In this case one *MU* is going to be moved from the *Transporter* to the workstation. If the *Transporter* is empty, nothing happens.

```
if @.NumMU > 0 // parts here?  
    @.Cont.move(workstation)  
end
```

See also

[else \[SimTalk\]](#)

Edit ribbon tab of the *Method* > **Select Template** > [if-else](#)

Edit ribbon tab of the *Method* > **Select Template** > [if-else-end](#)

Multiple Branching with if-elseif-end

At times a simple distinction between *true* and *false* will not suffice for handling all modeling situations. Then, you can use the `elseif` statement to select one of several possibilities.

Remarks

SimTalk analyzes the conditions one after the other until one condition is *true*. Then it executes the respective statement. If no condition is *true*, *SimTalk* executes the optional *else branch*. If there is no *else branch*, *Plant Simulation* continues after the final *end*.

Syntax

```
if condition1
  statement_list1
elseif condition2
  statement_list2
[... ]
[else
  statement_list3 ]
end
```

Example

```
// Is this part destined for us?
if transporter.destination = current
// yes, look for available workstation
  if Station_1.empty
    transporter.Cont.move(Station_1)
  elseif Station_2.empty
    transporter.Cont.move(Station_2)
  elseif Station_3.empty
    transporter.Cont.move(Station_3)
  else // no idle Station
    transporter.move(current.succ(1))
  end
else // the shipment is not for us
  transporter.move(current.succ(1))
end // end of multiple branching
```

See also

[else \[SimTalk\]](#)

[elseif \[SimTalk\]](#)

Edit ribbon tab of the *Method* > **Select** > **Template** > **if-elseif-end**

Conditional Expressions with when-then-else

The when-then-else-statement selects an expression based on the value of a *boolean* expression.

Syntax

```
when condition then expression1 else expression2
```

The syntax description reads like this:

- The statement `condition` is a boolean expression.
- The statement `expression1` is the expression that is to be selected if the `condition` is *true*.
- The statement `expression2` is the expression that is to be selected if the `condition` is *false*.

Examples

```
@.color := when ?.EntranceLocked then "red" else "green"
```

```
Method(x, y + (when dy > 0 then y+dy else 100))
```

See also

[when \[SimTalk\] - then \[SimTalk\] - else \[SimTalk\]](#)

Check for a Large Amount of Different Values with switch [SimTalk]

The control structure `switch` enables *Plant Simulation* to make easier and clearer choices when being confronted with several possible choices. This way you do not have to use lengthy *if-elseif-end-chains*.

Syntax

```
switch expression
  case constant_list statement_list
  [case constant_list statement_list ...]
  [else statement_list]
end
```

Examples

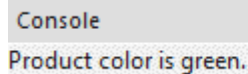
```
param number: integer
switch number
  case 1
    print "not a prime number"
  case 2,5,7,3
    print "prime number"
  case 9,4
    print "square number"
  else
    print "no special number"
    print "or number greater than 9"
end
```

```
var currentDay := 3
switch currentDay
  case 1
    print "Monday"
  case 2
    print "Tuesday"
  case 3
    print "Wednesday"
  case 4
    print "Thursday"
  case 5
    print "Friday"
  case 6
    print "Saturday"
  case 7
    print "Sunday"
end
```

Values of Data Type String

```
var ProductColor : string := "Green"
switch strToLower(ProductColor)
case "green"
    print "Product color is green."
case "blue"
    print "Product color is blue."
end
```

For **values of data type string** the *switch instruction* distinguishes between upper and lower casing.



Console
Product color is green.

Values of Data Type Real

For **values of data type real** the *switch instruction* does not check for exact equivalence.

For *if instructions* you can decide if you want to check for exact equivalence or if you want to permit a tolerance for **epsilon** by using = (equal to) or ~= (about equal).

switch instructions do not support this. As the epsilon value permits comparatively great differences, we decided to not use it for *switch instructions*. Instead, we ignore the last significant digit of the internal floating point display which will be rounded while displaying it and which you thus normally cannot see.

```
var x := 1.00000000000000004

print x      // outputs 1
print x - 1  // outputs 4.44089209850063e-16

if x = 1.0
    print "x = 1.0"
elseif x ~= 1.0
    print "x ~= 1.0" // will be reached
else
    print "x /= 1.0"
end

switch x
case 1.0
    print "case 1.0" // will be reached
else
    print "x /= 1.0"
end
```


Range of Definition

The term `expression` can be as complex as you need it to be. The data type of the result can be *integer*, *real*, or *string*.

Course of Execution

SimTalk analyzes the expression one time and converts it into the data type of the constant. If this is not possible, *Plant Simulation* shows an error message. It then executes the *case* branch, whose list of constants contains the result of the expression.

If no constant fits and the *switch instruction* contains an *else* branch, it executes the source code of the *else* branch.

The order of the *when-statements* and the order of the constants in the *when-statements* does not affect the speed with which the *switch instruction* is being executed.

SimTalk

[switch \[SimTalk\]](#)

[case \[SimTalk\]](#)

[Tolerance For About Equal \(~=\) Comparison \[model settings\]](#)

[setEpsilon \[SimTalk\]](#)

See also

Edit ribbon tab of the *Method* > **Select Template** > [switch-case-end](#)

Edit ribbon tab of the *Method* > **Select Template** > [switch-case-else-end](#)

Relational Operators

Loops

Loops [SimTalk]

Plant Simulation provides the `while`, the `repeat`, and the `for` loop.

- [while loop \[SimTalk\]](#)
- [repeat loop \[SimTalk\]](#)
- [for loop \[SimTalk\]](#)

You can **Exit a Loop** with `exitLoop`.

while loop [SimTalk]

Plant Simulation executes the instruction within the *while-loop* several times as long as the condition is fulfilled. *Plant Simulation* checks the condition before each loop iteration. If the condition is not even fulfilled on the first iteration, the instructions within the loop will not be executed at all.

Remarks

If the condition never results in the value *false*, the loop will be executed infinitely. Press **Ctrl+Alt+Shift** to stop the infinite loop and to open the *Method-Debugger*.

Examples

```
param n: integer -> integer // computes the factorial of n
result := 1
while n > 1
    result := n * result
    n -= 1
end
```

```
var n := 2
while n < 10
    n := n * 2
end
```

See also

Edit ribbon tab of the *Method* > **Select Template** > [while-end](#)

[Exit a Loop with exitLoop](#)

[while \[SimTalk\]](#)

[repeat loop \[SimTalk\]](#)

repeat loop [SimTalk]

Plant Simulation executes the instruction within the *repeat-loop* one or more times. *Plant Simulation* checks the condition after each loop iteration and only exits the loop when the condition is fulfilled. For this reason the loop will always be executed at least once.

Remarks

If the condition never returns *true*, the loop will be executed infinitely. Press **Ctrl+Alt+Shift** to stop the infinite loop and to open the *Method-Debugger*.

Example

```
var x: length
repeat
  x := methodEnterLength
until x >= 0
```

See also

Edit ribbon tab of the *Method* > **Select Template** > [repeat-until](#)

[Exit a Loop with exitLoop](#)

[repeat \[SimTalk\]](#)

[while loop \[SimTalk\]](#)

for loop [SimTalk]

The *for-loop* iterates through a range between a start value and an end value.

Remarks

The *for loop* declares a *loop variable* of data type *integer*. The *loop variable* is initialized with a start value. At the end of the loop, *Plant Simulation* automatically increases the *loop variable* by 1. The loop is executed as long as the *loop variable* is less than or equal to the end value. If the end value is less than the start value, *Plant Simulation* does not execute the loop at all.

The *loop variable* is only visible within the loop. After the loop is executed, you cannot access the *loop variable* anymore.

Note

For performance reasons *Plant Simulation* evaluates the start value and end value only once!

```
for var y := 1 to table.yDim -- assignment with the := operator
  print table[1,y]
next
```

```
for var y = 1 to table.yDim -- assignment with the = operator
  print table[1,y]
next
```

Use the keyword *downto* instead of the keyword *to* to iterate through a range backward. *Plant Simulation* reduces the loop variable by 1 each time. *Plant Simulation* executes the loop as long as the *loop variable* is greater than or equal to the end value. If the end value is greater than the start value, *Plant Simulation* does not execute the loop at all.

```
for var y := table.yDim downto 1
  print table[1,y]
next
```

The *loop variable* is only visible within the loop. Outside of the loop the identifier instead points to an object with this name if this object exists, for example to a *global variable*.

```
// the loop variable 'i' is only visible inside of the loop
for var i := 1 to ParallelStation.NumMU
  print ParallelStation.MU(i)
next

print i // the loop variable is not visible here, consequently 'i' must be
the name of an object
```

```
// another loop with its own loop variable 'i'
for var i := ParallelStation.NumMU downto 1
  print ParallelStation.MU(i)
next

// the local variable 'i' is visible from here to the end of the source code
var i := 1.234
print i // prints 1.234

for var i := 1 to 10 // does not compile, because 'i' is already declared
next
```

You can also use a *local variable* of data type *integer* as the *loop variable*. Then omit the keyword `var` or `local` respectively.

```
var y: integer
for y := Table.yDim downto 1
  if Table[1,y] > 100
    exitloop
  end
next
print y // outputs the index of the first table value greater than 100
```

See also

Edit ribbon tab of the *Method* > **Select Template** > [for-next](#)

[Exit a Loop with exitLoop](#)

[for \[SimTalk\]](#)